

1) Conceptos básicos

- Algoritmo: secuencia ordenada y no ambigua de pasos que resuelve un problema. Debe ser preciso, definido y finito; es independiente del lenguaje. Estructura: Entrada - Proceso - Salida.
- Instrucción: tarea elemental. Programa: conjunto de instrucciones que resuelven un problema.
- Lenguaje de programación: normas que permiten escribir programas comprensibles por el ordenador.
- Subprograma: fragmento que resuelve un subproblema. Favorece reusabilidad y modularidad.

2) Función, procedimiento y traductor

- Función: subprograma con n entradas y una salida.
- Procedimiento: subprograma con n entradas y n salidas según la terminología del tema; en la explicación posterior se caracteriza normalmente por realizar una acción y no devolver un valor directo.
- Traductor: procesa texto fuente y genera texto objeto. Tipos principales: compilador e intérprete.

3) Compilador vs intérprete

Compilador	Intérprete
Traduce código fuente a código objeto/intermedio	Ejecuta el fuente sin generar programa objeto
Traducción previa a la ejecución	Traducción a medida, normalmente instrucción a instrucción
Ejecución posterior más rápida	Mayor sobrecarga en tiempo de ejecución

Compilación: análisis léxico -> sintáctico -> semántico; síntesis: generación y optimización de código.

4) Etapas de compilación

- 1. Preprocesado: interpreta directivas.
- 2. Compilación: transforma a ensamblador.
- 3. Ensamblado: genera código objeto.
- 4. Enlazado: une módulos y bibliotecas.
- Enlace estático: bibliotecas dentro del ejecutable. Dinámico: se cargan en ejecución.
- Cross compiler: genera código para otra plataforma. Transpilador: traduce entre lenguajes del mismo nivel (ej. TypeScript -> JavaScript).

5) Clasificación por abstracción y propósito

Criterio	Clases / ejemplos
Abstracción	Máquina -> bajo (ensamblador) -> medio -> alto nivel
Propósito	General; específico; sistemas; script
General	C, C++, C#, Java, Visual Basic
Específico	COBOL gestión; Fortran científico; SQL BD; Prolog/Lisp IA
Sistemas	C
Script	sh, bash, dash, ksh, zsh, csh, tcsh

6) Paradigmas principales

- Imperativo/procedimental: describe cómo realizar el cálculo; secuencia, selección y repetición. Ej.: C, Pascal, BASIC, ADA, COBOL, FORTRAN.
- POO: objetos y clases; atributos y métodos. Conceptos: abstracción, encapsulamiento, ocultación, polimorfismo y herencia.
- Funcional: funciones puras, transparencia referencial y sin efectos colaterales; la repetición suele resolverse con recursividad. Ej.: Haskell.
- Lógico: reglas + hechos -> resultados. Ej.: Prolog. Declarativo: describe qué obtener, no cómo. Ej.: SQL.

7) Otros paradigmas y clasificación

- AOP: modulariza aspectos transversales; términos: aspecto, advice y pointcut.
- Dirigida por eventos: la ejecución responde a eventos mediante manejadores.
- Restricciones: se especifican variables y restricciones; un resolutor busca soluciones.
- Reactiva: programación asíncrona y propagación de cambios; patrón Observer.
- Reflexiva: el programa examina/modifica su estructura en ejecución. Concurrente: procesos/hilos simultáneos.
- Traducción: compilados (C), interpretados (JavaScript), mixtos (Java: bytecode + máquina virtual).
- Generaciones: 1GL máquina; 2GL simbólicos; 3GL medio/alto; 4GL específicos/declarativos (SQL); 5GL IA.

8) Tipado de lenguajes

- Fuertemente tipado: limita conversiones implícitas; ejemplos del tema: C, C++, Java, Python, Haskell, ML.
- Débilmente tipado: más permisivo con conversiones; PHP, JavaScript, Lisp, Prolog, Perl.
- Estático: la variable se declara con tipo; típico de compilados (Java, C#).
- Dinámico: no suele declararse el tipo de la variable; el tipo corresponde al contenido y puede cambiar (Python, PHP).

9) Tipos primitivos (Java)

Grupo	Tipos / tamaño
Enteros	byte 8; short 16; int 32; long 64 bits
Carácter	char 16 bits, Unicode
Decimales	float 32; double 64 bits, IEEE 754
Lógico	boolean: true / false

Tipos predefinidos: entendidos por el compilador. Tipos abstractos: definidos por el usuario, normalmente mediante clases.

10) Conversión de tipos / casting

- Implícita: asignación de tipo más pequeño a uno más grande sin notación explícita.
- Explícita: se fuerza la conversión aceptando posible pérdida de precisión.
- Formato: (tipo) valor_a_convertir.
- Ejemplo de ampliación del tema: byte -> short -> int -> long y float -> double.

11) Operadores

Tipo	Operadores
Aritméticos	+ - * / % ++ --
Relacionales	< <= == > >= !=
Lógicos	&& !
Ternario	condición ? si_true : si_false
Bits	& >> << ^ ~
Asignación	= += -= %= *= /= ^= >>= <<= &= =

12) Precedencia general

- 1 () [] . -> 2 ++ -- ! -> 3 * / % -> 4 + -
- 5 > >= < <= -> 6 == != -> 7 && -> 8 ||
- 9 ?: -> 10 = += -= *= /= %=
- Secuencias de escape destacadas: \n nueva línea, \t tabulador, \\ barra invertida, comillas escapadas.

13) Condicionales

- if: ejecuta un bloque si la condición booleana es true. Sin llaves, sólo la instrucción inmediata pertenece al if.
- if-else: bloque alternativo cuando la condición es false.
- if anidado / else if: selección encadenada.
- switch-case: selección múltiple según el valor de una expresión; suele usar break y puede incluir default.

14) Bucles

Bucle	Clave
while	Control de entrada; puede ejecutarse 0 veces
for	Inicialización + condición + incremento/decremento
for-each	Recorre colección/array secuencialmente sin necesitar índice
do-while	Control de salida; ejecuta el cuerpo al menos 1 vez

15) Recursividad

- Define un problema en base a su propia definición: divide un problema complejo en subproblemas del mismo tipo y menor tamaño.
- Necesita caso base o de parada. Técnica: divide y vencerás.
- Inconvenientes: puede ser más lenta por sobrecarga de llamadas y provocar stack overflow si hay demasiadas llamadas.

16) Funciones y procedimientos

- Son bloques reutilizables o subrutinas que pueden invocarse sin reescribir su código.
- Función: normalmente devuelve un valor y puede formar parte de una expresión.
- Procedimiento: realiza una acción y su nombre aparece como instrucción independiente.
- Función: TipoDatoDev nombreFuncion(TipoDato1 p1, ...).
- Procedimiento: void nombreProcedimiento(...).
- Parámetros formales: definidos en la cabecera. Actuales: valores concretos pasados al invocar.

17) Ámbito y paso de parámetros

- Global: definida fuera de funciones/procedimientos; visible en el código posterior según el tema.
- Local: definida dentro de una subrutina; sólo visible dentro de ella.
- Por valor: se pasa una copia; la rutina no puede modificar la variable real.
- Por referencia: se pasa una referencia/dirección; la rutina puede modificar la variable real.
- return: devuelve el resultado de una función o transfiere el control al llamador.

18) Vectores / arrays

- Array: colección de variables del mismo tipo bajo un nombre común; puede tener una o más dimensiones.
- Declaración Java: tipo nombre[]; o tipo[] nombre;. Reserva: nombre = new tipo[tamaño];.
- Índices: desde 0 hasta tamaño - 1.
- Inicialización automática al crear con new: 0 para numéricos, false para boolean y null para referencias.
- Matrices: arrays de arrays, p. ej. int[][] a = new int[10][20];.

19) Estructura de un programa

Bloque	Función
Entrada	Obtiene datos de usuario, periféricos, BD, sensores, otras aplicaciones...
Procesamiento	Realiza el cálculo o tarea principal
Salida	Guarda o envía el resultado: BD, periférico, actuador, otro programa, pantalla

20) Ojo de examen

- while puede ejecutarse 0 veces; do-while al menos 1.
- == compara; = asigna. &&/|| son lógicos; &/|^/~ operan a nivel de bits.
- Preincremento modifica antes de usar el valor; postincremento usa primero y modifica después.
- Por valor no altera el argumento real; por referencia sí puede.
- Array: mismo tipo, índice desde 0. Función: devuelve valor; procedimiento: acción.